CASE STUDY

Diffed.gg

A two-sided gaming services marketplace — coaching, boosting, real-time team formation.

ROLE
Lead full-stack engineer (contract)

LIVE AT
diffed.gg

STATUS
Live · Built end-to-end for the founder

STACK
Next.js 15 · React 19 · TypeScript · Prisma · PostgreSQL · Socket.IO · Stripe · PayPal · NextAuth/JWT · UploadThing · Vercel Blob · Nodemailer

Uzair Saleem · uzairsaleem.dev

The brief

> **TL;DR**
>
> Diffed.gg is a two-sided marketplace that connects gamers with vetted coaches and boosters across multiple titles. Customers configure a service (coaching session, duo queue, ELO-based boost), pay securely, and assemble a team of providers who apply to the order in real time. Providers earn through an in-platform wallet; admins curate the catalog, verify completed work, and process payouts.
>
> The platform handles the full transaction lifecycle — discovery, checkout, team formation, real-time chat, proof-of-completion, verification, and payout — under a single product.

The problem

Gaming service marketplaces tend to fail in three places: **trust** (is this booster legit?), **coordination** (how do I actually talk to my team mid-order?), and **payouts** (when and how do providers get paid fairly?). Most existing tools push these problems off-platform — into Discord DMs, Wise transfers, and screenshots in a ticket queue. Diffed.gg needed to bring all of that inside one product without sacrificing the live, queue-driven feel that providers expect.

What I built

Customer flow

- Browse a catalog of games → services → subpackages with dynamic pricing (per-game, per-teammate, ELO-tier based).
- Stripe and PayPal checkout, both server-side validated against the configured price.
- Post-payment, the order enters a public queue where providers apply. The customer accepts/declines applicants until team size is met, with a bounded reroll budget to prevent abuse.
- Real-time chat with the assembled team, post-completion reviews, and tipping.

Provider flow

- Application → admin activation → live queue.
- Auto-refreshing queue with diffed updates: skeleton on first load, seamless polling after, with toast + sound on new orders so providers stay aware without staring at the page.
- Wallet, transaction history (normalized 2-decimal currency), and admin-mediated payout requests.

• Screenshot proof-of-completion via UploadThing.

Admin flow

• Catalog CRUD (games, services, subpackages, ELO pricing tiers).

• Order verification — single click credits the provider wallet with the platform fee (20%) deducted and splits the remainder across all assigned providers.

• Payout queue, customer/provider management, and email-based admin invites with one-time tokens, expiry, and a registration handoff that promotes the invitee to admin on completion.

Key technical decisions

• **Next.js App Router + Server Actions** for the bulk of mutations, with REST-style route handlers reserved for things needing external callers (Stripe/PayPal webhooks, UploadThing).

• **Prisma + PostgreSQL** with a normalized schema spanning Users, Orders, Assignments, Subpackages, Wallets, Transactions, Reviews, and Invites. Earnings and payouts are modeled as immutable transaction rows so balances are always derivable.

• **Socket.IO** for chat and live order events. The provider queue uses gentle polling rather than sockets — it tolerates a few seconds of staleness and avoids the connection-pressure cost of pushing every queue mutation to every online provider.

• **Payment integrity** — server recomputes the price from the subpackage + configuration on every checkout intent so a tampered client price never reaches the gateway.

• **Auth** with JWT sessions via `jose` , role-gated layouts (customer / provider / admin), invite-token flow for admin onboarding.

• **Email** through Nodemailer (Gmail SMTP in dev, transactional provider in prod) for password resets and admin invites.

Hardest parts

1. Team assembly UX

Orders need N providers, applicants arrive asynchronously, and customers can decline within a reroll budget. Getting the state machine right — pending → partial team → complete → in-progress → verified — without dead-ends took several iterations.

2. Money correctness

Splitting an order across multiple providers, deducting the 20% fee, and rendering everything as fixed-2-decimal strings everywhere was deceptively annoying. **Floats were banished early; everything is integer cents in the DB and formatted at the edges.**

3. Real-time without the cost

Sockets for chat, polling for queues, and a small notification primitive that plays sound and shows a toast — even when the user is on a different page within the dashboard.

Outcomes

• Single product covering the full marketplace lifecycle — no off-platform Discord/Wise dependency.

• Sub-second perceived latency on the provider queue thanks to skeleton-on-first-load + silent polling.

• Admin onboarding reduced from a manual DB toggle to a one-click email invite.

• Clean separation between customer, provider, and admin layouts — new role-specific features ship without regressions.

What I'd do next

- Move queue updates to Socket.IO rooms keyed by game/service to drop the polling fallback entirely.
- Introduce a dispute/escrow state between “submitted proof” and “verified” so customers have a formal challenge window.
- Provider performance analytics (acceptance rate, completion time, repeat-customer rate) in the admin panel.

Stack

Next.js 15	React 19	TypeScript	Prisma	PostgreSQL	Socket.IO
Stripe	PayPal	NextAuth/JWT	UploadThing	Vercel Blob	Nodemailer